

AI-Powered Multi-Agent Energy Research System

A Project-II Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING**

BY

Ansh Upadhyay - EN22CS301147

Anuroop Patidar - EN22CS301176

Arnika Jain - EN22CS301196

Arnoday Soni - EN22CS301197

Aryan Singh - EN22CS301214

Under the Guidance of

Prof. Akshay Saxena

Prof. Hemlata Patel

Prof. Ajeet Singh Rajput



MEDICAPS
U N I V E R S I T Y

Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

JAN-JUNE 2026

AI-Powered Multi-Agent Energy Research System

A Project-II Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING**

BY

Ansh Upadhyay - EN22CS301147

Anuroop Patidar - EN22CS301176

Arnika Jain - EN22CS301196

Arnoday Soni - EN22CS301197

Aryan Singh - EN22CS301214

Under the Guidance of

Prof. Akshay Saxena

Prof. Hemlata Patel

Prof. Ajeet Singh Rajput



MEDICAPS
U N I V E R S I T Y

Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

JAN-JUNE 2026

Report Approval

The project work “**AI-Powered Multi-Agent Energy Research System**” is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approved any statement made, opinion expressed, or conclusion drawn there in; but approve the “Project Report” only for the purpose for which it has been submitted.

Internal Examiner

Name: Prof. Akshay Saxena

Prof. Hemlata Patel

Prof. Ajeet Singh Rajput

Designation: Faculty of Engineering, Medicaps University Indore

Affiliation

Declaration

We hereby declare that the project entitled “**AI-Powered Multi-Agent Energy Research System**” submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and engineering completed under the supervision of **Prof. Akshay Saxena, Prof. Hemlata Patel, Prof. Ajeet Singh Rajput**, Faculty of Engineering, Medicaps University Indore is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date

Ansh Upadhyay - EN22CS301147

Anuroop Patidar - EN22CS301176

Arnika Jain - EN22CS301196

Arnoday Soni - EN22CS301197

Aryan Singh - EN22CS301214

Certificate

We, **Prof. Akshay Saxena, Prof. Hemlata Patel, Prof. Ajeet Singh Rajput** certify that the project entitled “**AI-Powered Multi-Agent Energy Research System**” submitted in partial fulfillment for the award of the degree of Bachelor of Technology of Computer Applications by **Ansh Upadhyay, Anuroop Patidar, Arnika Jain, Arnoday Soni, Aryan Singh** is the record carried out by them under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Prof. Akshay Saxena

Computer Science & Engineering
Medicaps University, Indore

Prof. Hemlata Patel

Computer Science & Engineering
Medicaps University, Indore

Prof. Ajeet Singh Rajput

Computer Science & Engineering
Medicaps University, Indore

Dr. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

Arnika Jain – GenAI, Agentic AI, DevOps Certificates





Aryan Singh – GenAI, Agentic AI, DevOps Certificates



nasscom



CERTIFICATE OF COMPLETION

Awarded to

Aryan Singh

has successfully completed the Skill-Based Project comprising 80 hours of structured learning and practical training in **Agentic AI**, conducted from

February 28, 2026 to March 30, 2026

The candidate successfully completed the skill based project with satisfactory performance.

We wish him/her continued success.

Mr. Dhaval Shah
Chief Executive Officer

Certificate number: EDU-2026-K2W72



nasscom



CERTIFICATE OF COMPLETION

Awarded to

Aryan Singh

has successfully completed the Skill-Based Project comprising 80 hours of structured learning and practical training in **DevOps Foundation**, conducted from

April 01, 2026 to May 01, 2026

The candidate successfully completed the skill based project with satisfactory performance.

We wish him/her continued success.

Mr. Dhaval Shah
Chief Executive Officer

Certificate number: EDU-2026-6VY6V



Anuroop Patidar – GenAI, Agentic AI, DevOps Certificates





Arnoday Soni – GenAI, Agentic AI, DevOps Certificates



nasscom



CERTIFICATE OF COMPLETION

Awarded to

Arnoday Soni

has successfully completed the Skill-Based Project comprising 80 hours of structured learning and practical training in **Agentic AI**, conducted from

February 28, 2026 to March 30, 2026

The candidate successfully completed the skill based project with satisfactory performance.

We wish him/her continued success.

Mr. Dhaval Shah
Chief Executive Officer

Certificate number: EDU-2026-9C9HA



nasscom



CERTIFICATE OF COMPLETION

Awarded to

Arnoday Soni

has successfully completed the Skill-Based Project comprising 80 hours of structured learning and practical training in **DevOps Foundation**, conducted from

April 01, 2026 to May 01, 2026

The candidate successfully completed the skill based project with satisfactory performance.

We wish him/her continued success.

Mr. Dhaval Shah
Chief Executive Officer

Certificate number: EDU-2026-5GX7T



Ansh Upadhyay – GenAI, Agentic AI, DevOps Certificates



nasscom



CERTIFICATE OF COMPLETION

Awarded to

Ansh Upadhyay

has successfully completed the Skill-Based Project comprising 80 hours of structured learning and practical training in **DevOps Foundation**, conducted from

April 01, 2026 to May 01, 2026

The candidate successfully completed the skill based project with satisfactory performance.

We wish him/her continued success.

A handwritten signature in black ink, appearing to read "DShah".

Mr. Dhaval Shah
Chief Executive Officer

Certificate number: EDU-2026-YMGF7



Acknowledgements

We would like to express my deepest gratitude to Honorable Chancellor, **Shri R C Mittal**, who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Mediacaps University, whose unfailing support and enthusiasm has always boosted up my morale. We also thank **Prof. (Dr.) Ratnesh Litoriya**, Associate Dean of computing & Allied Branches, Faculty of Engineering, Mediacaps University, for giving me a chance to work on this project. We would also like to thank my Head of Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

We express our heartfelt gratitude to our Internal Guide, Prof. Akshay Saxena, Prof. Hemlata Patel, Prof. Ajeet Singh Rajput without whose continuous help and support, this project would ever have reached to the completion.

Ansh Upadhyay - EN22CS301147

B.Tech. IV Year

Department of Computer Science & Engineering

Faculty of Engineering

Mediacaps University, Indore

Anuroop Patidar - EN22CS301176

B.Tech. IV Year

Department of Computer Science & Engineering

Faculty of Engineering

Mediacaps University, Indore

Arnika Jain - EN22CS301196

B.Tech. IV Year

Department of Computer Science & Engineering

Faculty of Engineering

Mediacaps University, Indore

Arnoday Soni - EN22CS301197

B.Tech. IV Year

Department of Computer Science & Engineering

Faculty of Engineering

Mediacaps University, Indore

Aryan Singh - EN22CS301214

B.Tech. IV Year

Department of Computer Science & Engineering

Faculty of Engineering

Mediacaps University, Indore

Abstract

The exponential growth of digital information in recent years has created both opportunities and challenges in the domain of knowledge discovery. While vast amounts of data are readily accessible through the internet, extracting meaningful insights from this data remains a time-consuming and complex task. Traditional research methods require manual effort to search, filter, analyze, and summarize information from multiple sources, making the process inefficient and prone to inconsistencies.

This project presents the design and implementation of an **AI-Powered Multi-Agent Energy Research and Report Generation System**, which automates the end-to-end research workflow using advanced Artificial Intelligence techniques. The system leverages **Generative Artificial Intelligence (GenAI)** and **Agentic AI architecture**, where multiple intelligent agents collaborate to perform specialized tasks such as web search, data extraction, content analysis, and report generation.

The proposed system integrates modern web technologies and machine learning frameworks to create a scalable and efficient research platform. The frontend is developed using React (Vite) to provide a responsive and user-friendly interface. The backend consists of a Node.js API server that acts as a middleware layer, and a Python-based FastAPI service that executes the core AI logic. The intelligence layer incorporates pre-trained transformer models, including Flan-T5 for text generation and Sentence Transformers for semantic embeddings.

To enhance information retrieval, the system utilizes **FAISS (Facebook AI Similarity Search)** for vector-based semantic search, enabling users to retrieve relevant information based on meaning rather than keyword matching. Additionally, structured research outputs are stored in JSON format, ensuring easy access and persistence of generated reports.

The application is designed following modern DevOps practices. It is containerized using Docker, orchestrated using Docker Compose, and deployed on AWS EC2. A CI/CD pipeline implemented through GitHub Actions automates the build, testing, and deployment process, ensuring continuous integration and reliable updates.

The developed system successfully demonstrates how AI, multi-agent systems, and cloud-based deployment can be integrated into a unified platform. It significantly reduces manual effort in research tasks, improves efficiency, and provides structured, high-quality outputs. The project highlights the practical application of emerging technologies in solving real-world problems and establishes a foundation for future enhancements in automated research systems.

Keywords:

Artificial Intelligence, Generative AI, Multi-Agent Systems, Web Scraping, Natural Language Processing, Semantic Search, FAISS, Docker, CI/CD, AWS EC2

Table of Contents

		Page No.
	Report Approval	ii
	Declaration	iii
	Certificate	iv - xii
	Acknowledgement	xiii
	Abstract	xiv
	Table of Contents	xv - xvi
	List of figures	xvii
	List of tables	xviii
	Abbreviations	xix
Chapter 1	Introduction (Whichever is applicable)	1-4
	1.1 Introduction	1
	1.2 Literature Review	2
	1.3 Objectives	2
	1.4 Significance	3
	1.5 Research Design	3
	1.6 Source of Data	4
	1.7 Chapter Scheme	4
Chapter 2	(Report on Present Investigation)	5-10
	2.1 System Architecture	5
	2.2 Technology Stack	7
	2.3 System Workflow	8
Chapter 3	Implementation	11-13
	3.1 Frontend Implementation	11
	3.2 Backend Implementation	12
	3.3 AI Agent Implementation	13
Chapter 4	DevOps & Deployment	14-15
	4.1 Dockerization	14
	4.2 CI/CD Pipeline (GitHub Actions)	14
	4.3 AWS EC2 Deployment	15
Chapter 5	Database & Storage	16
	5.1 FAISS Vector Database	16
	5.2 JSON Storage System	16

Chapter 6	Results and Discussions	17-19
	6.1 System Output	17
Chapter 7	Summary and Conclusions	20
	7.1 Summary	20
	7.2 Conclusion	20
Chapter 8	Future scope	21
	Appendix	22
	Bibliography	22

List of Figures

Figure No.	Title
Fig. 2.1	Overall System Architecture
Fig. 2.3	System Workflow Diagram
Fig. 6.1	User Interface – Query Input Page
Fig. 6.2	User Interface – Query Processing Page
Fig. 6.3	Generated Research Report Output (Part A)
Fig. 6.4	Generated Research Report Output (Part B)
Fig. 6.5	AWS EC2 Instance Dashboard
Fig. 6.6	Docker Containers Running

List of Tables

Table No.	Title
Table 1.1	Comparison of Traditional vs AI-Based Research Systems
Table 2.1	Technology Stack Overview
Table 3.1	API Endpoints Description
Table 5.1	JSON Report Structure
Table 6.1	Performance Analysis Metrics

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
GenAI	Generative Artificial Intelligence
LLM	Large Language Model
API	Application Programming Interface
UI	User Interface
UX	User Experience
CI/CD	Continuous Integration / Continuous Deployment
EC2	Elastic Compute Cloud
FAISS	Facebook AI Similarity Search
NLP	Natural Language Processing
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation

Chapter 1: Introduction

1.1 Introduction

In the contemporary digital age, the availability of information has grown at an unprecedented rate due to the widespread adoption of the internet, digital platforms, and open knowledge repositories. While this abundance of information provides immense opportunities for learning and research, it also introduces significant challenges in terms of data filtering, relevance identification, and knowledge synthesis. Users are often overwhelmed by the sheer volume of information and face difficulty in extracting meaningful insights efficiently.

Traditional research methodologies rely heavily on manual processes, where users search for information using search engines, browse multiple web pages, extract relevant data, and compile it into a structured format. This process is not only time-consuming but also requires domain expertise and critical thinking skills to ensure accuracy and completeness. Furthermore, the lack of automation in traditional systems leads to redundancy and inefficiency, especially when handling large datasets or repetitive research tasks.

With the advancement of Artificial Intelligence (AI), particularly in the field of Natural Language Processing (NLP), there has been a paradigm shift in how information is processed and utilized. Generative AI models, such as transformer-based architectures, have demonstrated the capability to understand, analyze, and generate human-like text. These models enable automated summarization, content generation, and intelligent decision-making, making them highly suitable for research applications.

The concept of **Agentic AI** further enhances this capability by introducing a multi-agent system where multiple intelligent agents collaborate to solve complex problems. Instead of relying on a single monolithic model, the system is divided into specialized components, each responsible for a specific task. This modular approach improves scalability, flexibility, and efficiency.

The **Energy Research App** is developed as an intelligent system that integrates Generative AI, multi-agent architecture, and modern web technologies to automate the research workflow. The system accepts user queries, retrieves relevant information from the web, processes and analyzes the data, and generates structured research reports. Additionally, it incorporates semantic search capabilities, allowing users to retrieve previously generated reports based on contextual similarity.

To ensure real-world applicability, the system is deployed using industry-standard DevOps practices. Docker is used for containerization, GitHub Actions for CI/CD automation, and AWS EC2 for cloud deployment. This not only ensures scalability and reliability but also demonstrates the practical integration of software engineering and AI technologies.

The system accepts a user-defined query related to energy or general research topics and performs the following operations:

- Searches the web for relevant information
- Scrapes content from multiple sources
- Filters and analyzes the collected data
- Generates a structured research report using a Large Language Model (LLM)

- Stores the results for future retrieval and semantic search

The application is designed using a **multi-layer architecture**, consisting of:

- A React-based frontend for user interaction
- A Node.js API server acting as a proxy
- A Python FastAPI backend handling AI processing
- A multi-agent system for research automation
- A vector database (FAISS) for semantic search

Additionally, the system is deployed using **Docker containers**, automated through **CI/CD pipelines using GitHub Actions**, and hosted on **AWS EC2**, demonstrating real-world DevOps practices.

This project not only automates research tasks but also showcases the integration of AI, full-stack development, and cloud deployment into a unified, scalable solution.

1.2 Literature Review

More recently, AI-based tools such as conversational agents and research assistants have emerged. These systems utilize Large Language Models (LLMs) to generate human-like responses and summaries. However, most existing solutions exhibit certain limitations:

- Lack of end-to-end automation of the research pipeline
- Limited ability to scrape and analyze real-time web data
- Absence of multi-agent collaboration mechanisms
- No persistent storage and semantic retrieval of research outputs

Research in Agentic AI systems has introduced the concept of dividing complex tasks into smaller subtasks handled by specialized agents.

1.3 Objectives

The primary objective of this project is to develop an intelligent system that automates the process of web-based research and report generation using advanced AI techniques.

The specific objectives are as follows:

- To design and implement an automated research system capable of retrieving and processing information from multiple web sources
- To develop a multi-agent architecture for efficient task distribution and execution
- To integrate Generative AI models for generating structured and coherent research reports
- To implement semantic search using vector embeddings and FAISS
- To create an intuitive and user-friendly frontend interface

- To ensure system scalability and portability through Docker-based containerization
- To implement a CI/CD pipeline for continuous integration and deployment
- To deploy the system on a cloud platform (AWS EC2) for real-world accessibility

1.4 Significance

The significance of this project lies in its ability to address real-world challenges in information retrieval and research automation. By integrating multiple advanced technologies, the system provides a comprehensive solution for efficient knowledge extraction and analysis.

The key contributions include:

- **Automation of Research Process:** Reduces manual effort and time required for gathering and analyzing information
- **Integration of AI Technologies:** Demonstrates practical use of Generative AI and Agentic AI
- **Scalable Architecture:** Designed using modular components for easy scalability and maintenance
- **Real-World Deployment:** Implements industry-standard DevOps practices such as containerization and CI/CD
- **Enhanced User Experience:** Provides structured and meaningful outputs instead of raw data

The system can be applied in various domains, including education, research, business intelligence, and data analysis.

1.5 Research Design

The system follows a modular and layered design approach, ensuring separation of concerns and ease of development.

Design Approach:

- **Frontend Layer:** Handles user interaction and displays results
- **API Layer:** Acts as a communication bridge between frontend and backend
- **Backend Layer:** Processes requests and manages AI operations
- **Agent Layer:** Executes specialized tasks such as research, analysis, and summarization
- **Storage Layer:** Stores reports and embeddings

The system workflow is designed as a pipeline:

1. User submits a query
2. Query is forwarded to backend services
3. AI agents process the query sequentially

4. Results are generated and stored
5. Output is returned to the user interface

This design ensures flexibility, scalability, and efficient processing of tasks.

1.6 Source of Data

The data used in this system is collected dynamically from online sources. The primary sources include:

- **Search Engines:** DuckDuckGo for retrieving relevant links
- **Web Pages:** Scraped using BeautifulSoup and HTTP requests
- **Fallback Sources:** Wikipedia for reliable information when search fails

The collected data is processed and filtered before being passed to the AI models for analysis and summarization.

1.7 Chapter Scheme

The report is organized into multiple chapters, each focusing on a specific aspect of the project:

- **Chapter 1:** Introduction, objectives, and overview of the system
- **Chapter 2:** System architecture, design, and workflow
- **Chapter 3:** Implementation details of frontend, backend, and AI agents
- **Chapter 4:** DevOps practices including Docker, CI/CD, and deployment
- **Chapter 5:** Database and storage mechanisms
- **Chapter 6:** Results and discussion of system performance
- **Chapter 7:** Summary and conclusions
- **Chapter 8:** Future scope and enhancements

Chapter 2: System Design & Investigation

2.1 System Architecture

The Energy Research App is designed using a modular, scalable, and microservices-based architecture that integrates frontend interaction, API handling, and AI-driven processing.

The system is divided into three major layers:

1. Presentation Layer (Frontend)

- Responsible for: Accepting user queries, Displaying research progress, Rendering structured reports
- Communicates with backend via HTTP APIs

2. Application Layer (API Server)

- Acts as a middleware/proxy layer
- Handles: Routing (/api/*), Request forwarding, CORS and security

3. Intelligence Layer (AI Backend)

- Implements Agentic AI architecture
- Consists of:
 - **Research Agent** → performs web search
 - **Scraper Tool** → extracts content
 - **Analysis Agent** → processes data
 - **Summary Agent** → generates final report using LLM

4. Data Layer

- FAISS → vector storage for semantic search
- JSON storage → stores generated reports
- Embeddings → Sentence Transformers

5. Deployment Layer

- Dockerized services
- Hosted on AWS EC2
- CI/CD via GitHub Actions

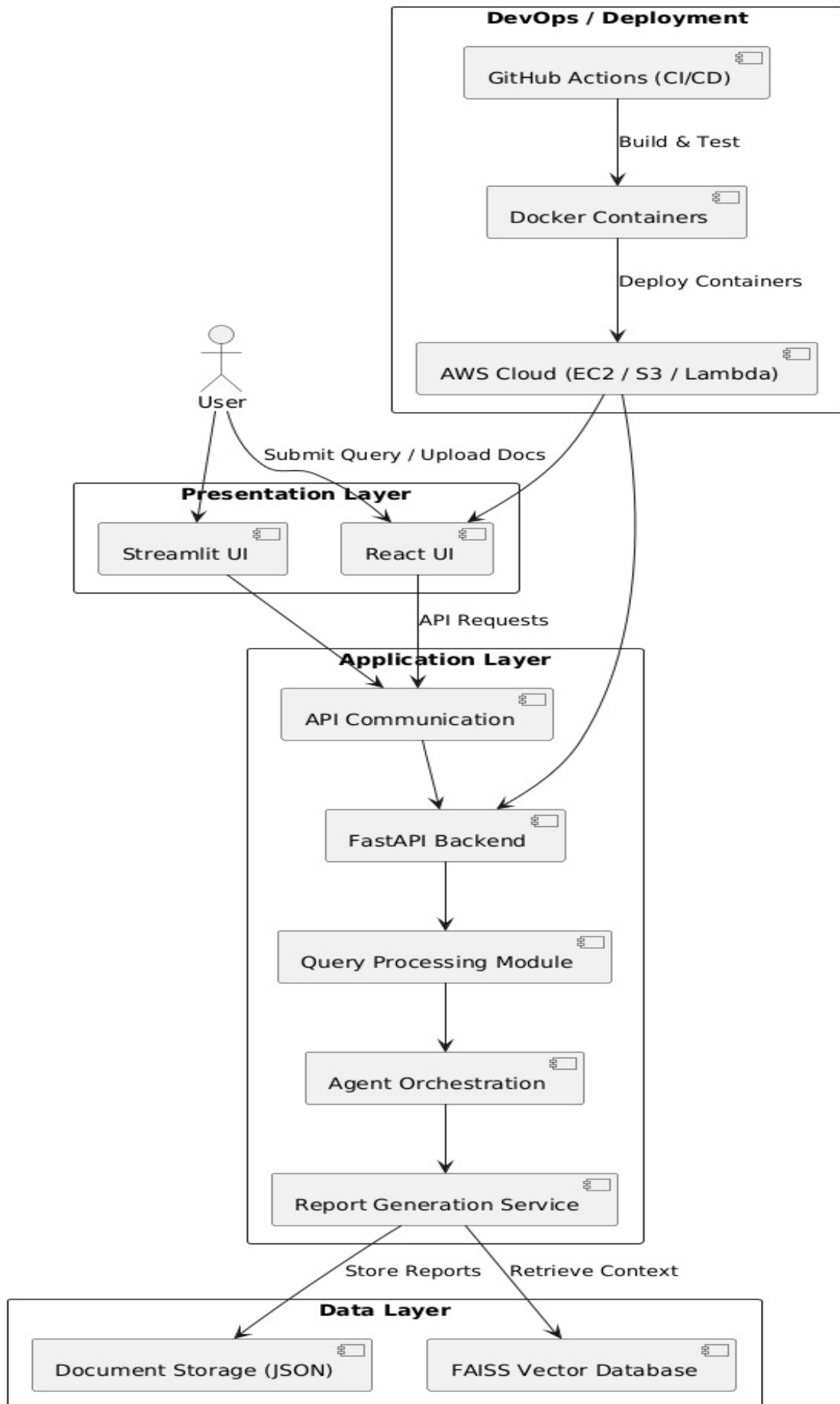


Fig. 2.1: Overall System Architecture

2.2 Technology Stack

The Energy Research App integrates a wide range of modern technologies across multiple layers, ensuring high performance, scalability, and ease of development.

2.2.1 Frontend Technologies

- **React (Vite):** Fast and modular UI development
- **TypeScript:** Type safety and improved maintainability
- **Tailwind CSS:** Utility-first styling framework
- **React Query:** Efficient data fetching and caching

Advantages:

- Faster development
- Responsive UI
- Improved state management

2.2.2 Backend Technologies

- **Node.js (Express):**
 - Acts as API gateway
 - Handles routing and proxy logic
- **FastAPI (Python):**
 - High-performance backend framework
 - Ideal for AI-based applications
 - Supports asynchronous processing

2.2.3 AI & Machine Learning Technologies

- **HuggingFace Transformers (Flan-T5):**
 - Generates structured summaries
 - Handles natural language understanding
- **Sentence Transformers:**
 - Converts text into embeddings
 - Used for semantic similarity
- **FAISS:**
 - High-speed vector similarity search
 - Efficient for large datasets

2.2.4 Data Processing Technologies

- **BeautifulSoup:**
 - HTML parsing
 - Web scraping
- **Requests Library:**
 - HTTP communication
- **DuckDuckGo Search:**
 - Fetches search results
 - Privacy-friendly search engine

2.2.5 DevOps & Deployment Technologies

- **Docker:** Containerization
- **Docker Compose:** Multi-container orchestration
- **GitHub Actions:** CI/CD pipeline
- **AWS EC2:** Cloud hosting

2.3 System Workflow

The system workflow represents the **end-to-end execution process**, from user query submission to report generation and display.

2.3.1 Workflow Overview

The system follows a **multi-stage pipeline architecture**, where each stage processes data and passes it to the next stage.

2.3.2 Step-by-Step Workflow

Step 1: User Input

- User submits a research query via frontend
- Example: *“What is renewable energy?”*

Step 2: API Request Handling

- Request is sent to API server (/api/research)
- API server forwards request to Python backend

Step 3: Research Agent Execution

- Performs web search using DuckDuckGo
- Retrieves relevant URLs

Step 4: Content Scraping

- Scraper extracts data from web pages
- Cleans and structures content

Step 5: Analysis Phase

- Analysis agent filters noise
- Extracts key insights and topics

Step 6: Report Generation

- Summary agent uses LLM (Flan-T5)
- Generates structured report:
 - Introduction
 - Key points
 - Conclusion

Step 7: Storage & Indexing

- Report stored in JSON format
- Embeddings generated and stored in FAISS

Step 8: Response Delivery

- Backend returns job ID
- Frontend fetches results using polling
- Final report displayed to user

2.3.3 Workflow Characteristics

- **Asynchronous Processing:** Improves responsiveness
- **Pipeline Architecture:** Efficient data flow
- **Modular Execution:** Each step independent
- **Scalable:** Can handle multiple requests

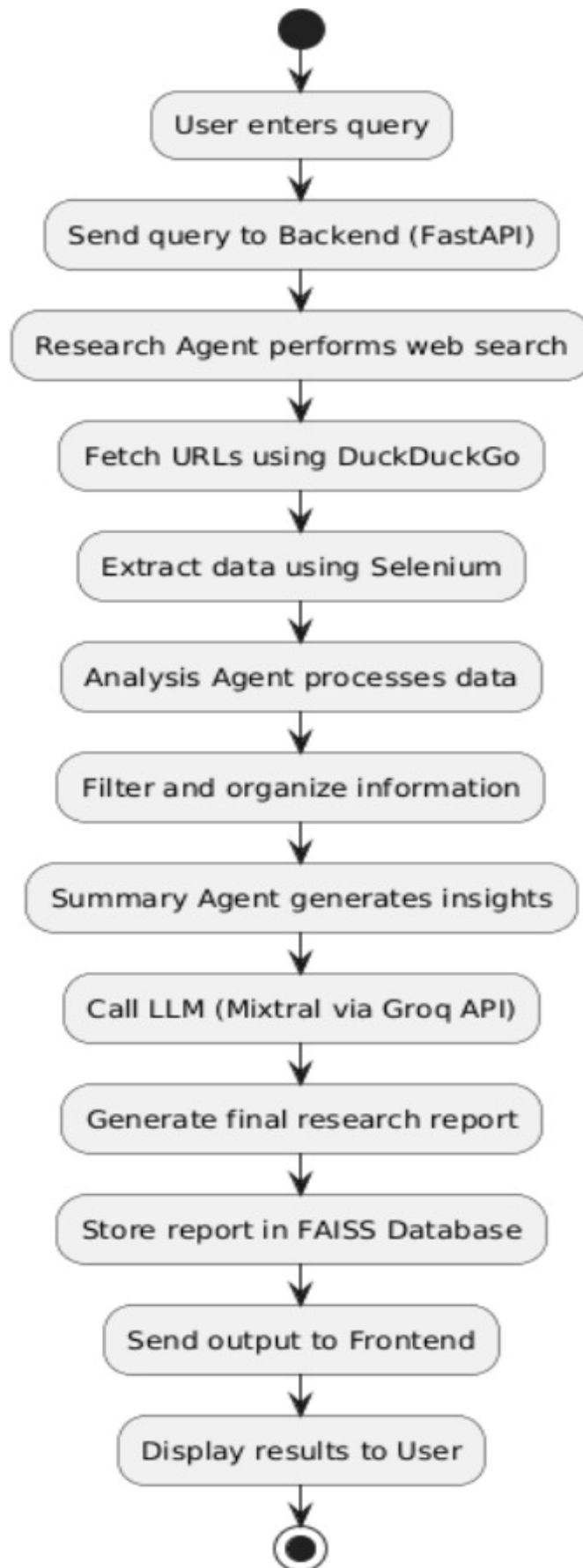


Fig. 2.3: System Workflow Diagram

Chapter 3: Implementation

3.1 Frontend Implementation

The frontend follows a **component-based architecture**, where the UI is divided into reusable components and pages.

Main Structure:

- Pages (Home, Job View, Report View, Search)
- Components (Sidebar, Layout, UI elements)
- API Layer (generated client + custom fetch)

This modular structure improves maintainability and scalability.

3.1.1 API Integration

The frontend communicates with backend services using a generated API client and a custom fetch wrapper.

- Base URL is configured using: `setBaseUrl("http://<EC2-IP>:8080");`
- All API calls are made using:
 - React Query (for caching & async handling)
 - Custom fetch wrapper for request control

3.1.2 Key Features Implemented

- **Query Input Interface**
 - Users enter research queries
 - Input validation is applied
- **Live Job Tracking**
 - Displays progress of research
 - Uses polling mechanism
- **Research History**
 - Displays previously generated reports
 - Allows quick access
- **Report Visualization**
 - Structured display of generated reports
 - Clean and readable format
- **Semantic Search**
 - Enables intelligent search over stored reports

3.2 Backend Implementation

The backend consists of two layers:

1. API Server (Node.js)
2. AI Backend (FastAPI)

3.2.1 API Server (Node.js - Express)

The API server acts as a gateway between frontend and AI backend.

Responsibilities:

- Route /api/* requests
- Forward to Python backend
- Handle CORS issues
- Maintain separation of concerns

Example: /api/research → forwarded to → Python /research

3.2.2 Python Backend (FastAPI)

The Python backend handles all AI processing and business logic.

3.2.3 API Endpoints Implemented

Method	Endpoint	Description
GET	/healthz	Health check
POST	/research	Start research
GET	/research/{id}	Get result
GET	/history	Fetch reports
POST	/search	Semantic search

3.3 AI Agent Implementation

The core intelligence of the system is based on a multi-agent architecture, where each agent performs a specific task in the research pipeline.

3.3.1 Research Agent

- Performs web search using DuckDuckGo
- Collects top URLs
- Initiates scraping process

3.3.2 Analysis Agent

- Filters irrelevant data
- Extracts key insights
- Identifies main topics

3.3.3 Summary Agent

- Uses HuggingFace Flan-T5 model
- Generates structured reports:
 - Introduction
 - Key insights
 - Conclusion

3.3.6 Agent Pipeline Flow

The agents operate sequentially:

1. Research Agent → gathers data
2. Scraper → extracts content
3. Analysis Agent → processes data
4. Summary Agent → generates report

Chapter 4: DevOps & Deployment

4.1 Dockerization

To ensure portability, consistency, and scalability, the entire application is containerized using **Docker**. Each major service runs inside its own isolated container.

4.1.1 Multi-Container Architecture

The system is divided into three Docker services:

- **Frontend Container**
 - Served via Nginx
 - Runs on port 80
- **API Server Container**
 - Handles routing and proxy logic
 - Runs on port 8080
- **Python Backend Container**
 - Executes multi-agent pipeline
 - Runs on port 8000

4.1.2 Docker Compose Orchestration

All services are managed using **Docker Compose**, enabling:

- Service networking
- Environment isolation
- Easy startup/shutdown

Example: services: frontend: api-server: python-backend:

4.2 CI/CD Pipeline (GitHub Actions)

The project implements a **Continuous Integration and Continuous Deployment (CI/CD)** pipeline using **GitHub Actions**.

4.2.1 Pipeline Overview: The pipeline automates:

- Code integration
- Docker image building
- Image publishing
- Remote deployment

4.2.2 Workflow Steps

1. Developer pushes code to GitHub (main branch)

2. GitHub Actions is triggered
3. Docker images are built:
 - energy-frontend
 - energy-api-server
 - energy-python-backend
4. Images are pushed to Docker Hub
5. EC2 instance is accessed via SSH
6. Latest images are pulled
7. Containers are restarted using Docker Compose

4.3 AWS EC2 Deployment

The application is deployed on a **cloud-based virtual server using AWS EC2**, enabling remote access and real-time usage.

4.3.1 EC2 Instance Setup

- OS: Ubuntu Linux
- Docker & Docker Compose installed

4.3.2 Deployment Process

1. EC2 instance is launched
2. SSH access configured using key pair
3. Docker environment installed
4. Application deployed via CI/CD pipeline
5. Services started using Docker Compose

4.3.3 Application Access

- Frontend accessed via: `http://<EC2-PUBLIC-IP>`
- API server: `http://<EC2-PUBLIC-IP>:8080`

4.4 Summary of DevOps Implementation

The integration of Docker, CI/CD, and AWS enables:

- Automated deployment pipeline
- Scalable architecture
- Reduced manual errors
- Faster development-to-production cycle

Chapter 5: Database & Storage

5.1 FAISS Vector Database

The system uses FAISS as a vector database to enable semantic search over stored research reports. Traditional databases rely on keyword matching, which is limited. FAISS allows:

- Semantic similarity search
- Faster retrieval of relevant information
- Efficient handling of high-dimensional embeddings

5.1.1 Working Principle

1. Text data is converted into vector embeddings using Sentence Transformers
2. These embeddings are stored in FAISS index
3. When a query is received:
 - It is converted into a vector
 - FAISS finds closest matching vectors using similarity search

5.2 JSON Storage System

In addition to FAISS, the system uses a JSON-based storage mechanism for storing complete research reports.

- Store structured research outputs
- Maintain history of generated reports
- Provide quick retrieval without recomputation

5.2.2 Storage Structure

Each report is stored as a JSON file containing: Report ID, Query, Generated summary, Sources, Timestamp

Example structure:

```
{
  "id": "report_id",
  "query": "What is renewable energy?",
  "summary": "...",
  "sources": ["url1", "url2"],
  "timestamp": "2026-04-21"
}
```

Chapter 6: Results and Discussions

6.1 System Output

The Energy Research App successfully generates structured research reports from user queries using its multi-agent pipeline.

6.1.1 Output Generation

When a user submits a query:

- The system processes it through multiple agents
- Extracts relevant information from web sources
- Generates a structured report containing:
 - Introduction
 - Key insights
 - Summary
 - References

6.1.2 Sample Output Description

For example, for the query:

“What is energy?”

The system produces:

- Definition and explanation
- Types of energy
- Real-world applications
- References from multiple sources

6.1.3 Frontend Output Display

The frontend displays:

- Research progress (live updates)
- Final structured report
- History of previous searches
- Semantic search results



Fig. 6.1: User Interface – Query Input Page

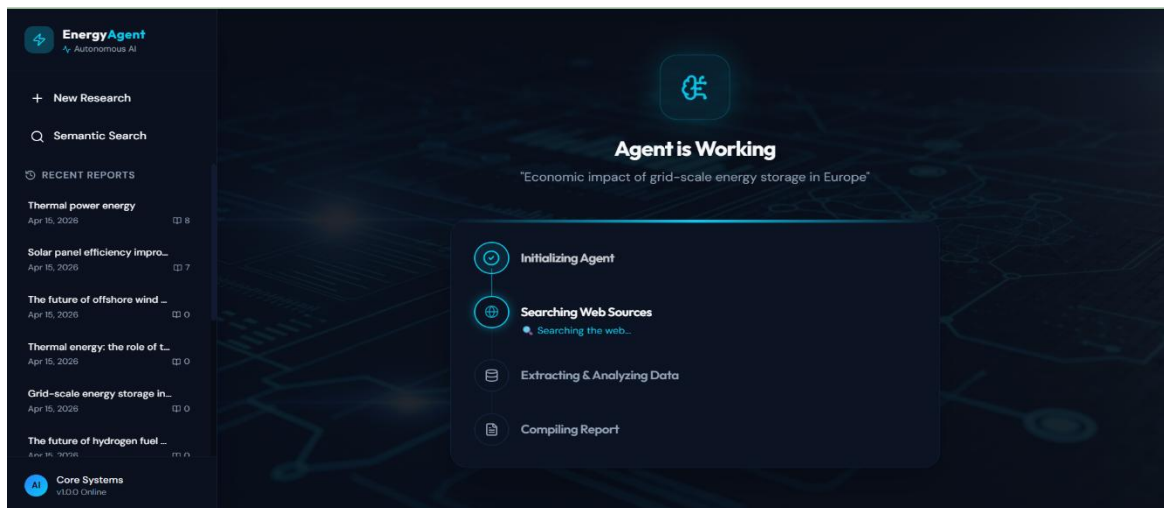


Fig 6.2: User Interface – Query Parsing Page



Fig 6.3: Generated Research Report Output Page (a)

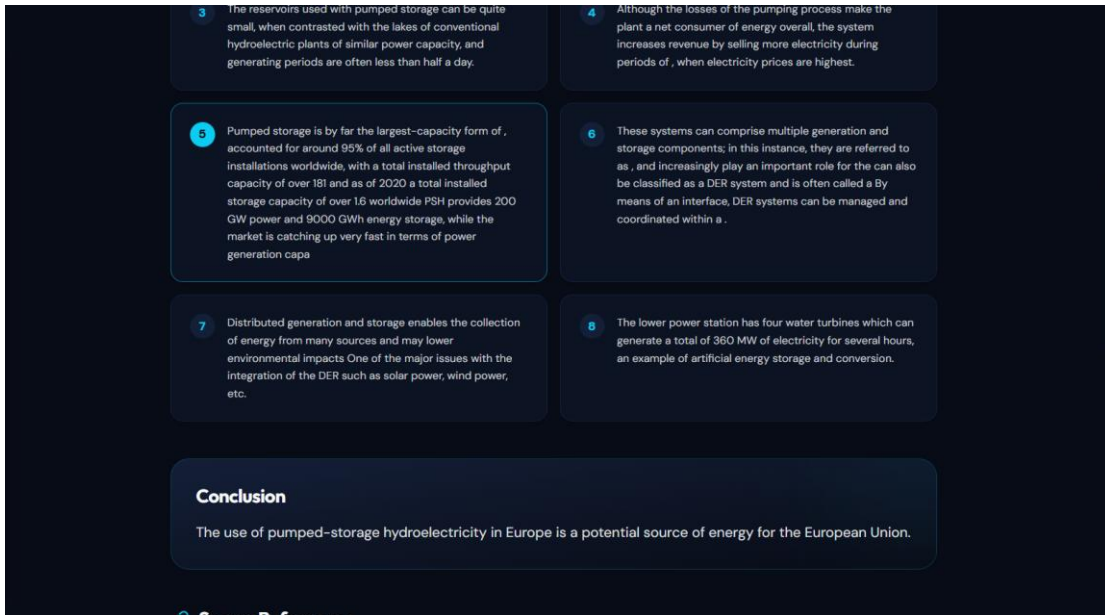


Fig 6.4: Generated Research Report Output Page (b)

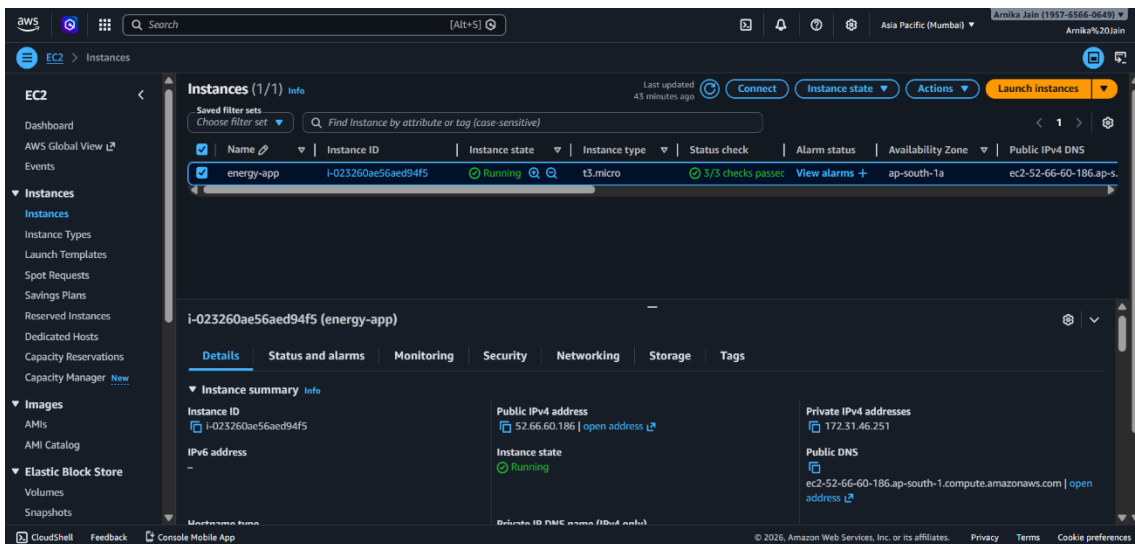


Fig 6.5: AWS EC2 Instance

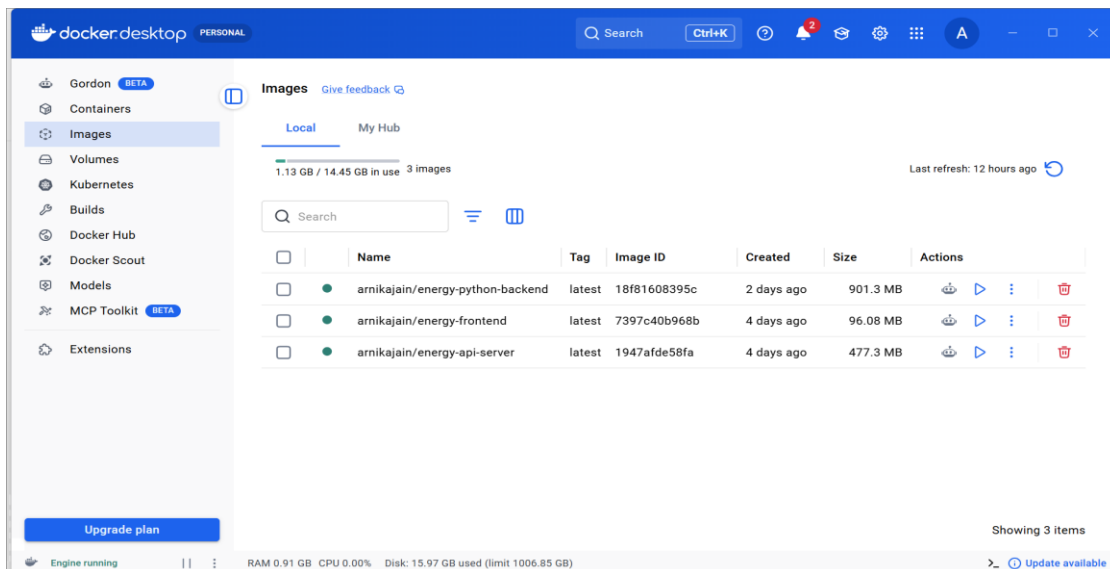


Fig 6.6: Docker Desktop

Chapter 7: Summary and Conclusion

7.1 Summary

The application follows a multi-agent architecture, where:

- The Research Agent collects data
- The Analysis Agent processes content
- The Summary Agent generates structured reports

Challenges Faced

During development and deployment, several technical challenges were encountered.

API Routing Issues:

Problem: 404 errors when frontend tried to access backend

Solution: Fixed API proxy configuration, Corrected base URL in frontend

Deployment Issues on Cloud:

Problem: Application worked locally but not on EC2

Solution: Updated Docker configuration, Corrected exposed ports and URLs

Disk Space Issues:

Problem: EC2 storage getting full due to multiple images

Solution: Implemented cleanup command: `docker image prune -af`

7.2 Conclusion

The Energy Research App successfully demonstrates how Artificial Intelligence, Agentic Systems, and DevOps practices can be integrated into a unified application.

Key conclusions drawn from the project:

- The system is capable of generating accurate and structured research outputs
- The multi-agent approach improves modularity and efficiency
- Semantic search using FAISS enhances data retrieval capabilities
- Dockerization and CI/CD enable smooth and automated deployment
- The system is scalable and production-ready

Despite minor challenges during development, such as API routing issues and deployment errors, the final system operates reliably and efficiently.

Overall, the project showcases a modern AI-driven application architecture combining Machine Learning, Web Technologies, Cloud Deployment, Automation (DevOps)

Chapter 8: Future Scope

The current system provides a strong foundation for automated research; however, several enhancements can be implemented to improve functionality and scalability.

1 Advanced AI Models

- Integration of more powerful LLMs (e.g., GPT-based or larger transformer models)
- Improved summarization quality and contextual understanding

2 Real-Time Data Integration

- Use of live APIs for real-time data sources
- Integration with academic databases (e.g., research papers)

3 User Authentication System

- Login and user-specific dashboards
- Personalized research history

4 Scalability Improvements

- Deployment using Kubernetes for container orchestration
- Load balancing for handling multiple users

5 Enhanced Semantic Search

- Hybrid search (keyword + vector search)
- Improved ranking algorithms

6 UI/UX Improvements

- Interactive dashboards
- Visualization of research data (charts, graphs)

7 Mobile Application

- Development of Android/iOS version
- Cross-platform support

8 Integration with External Tools

- Integration with citation tools
- Export reports as PDF or DOCX

9 Performance Optimization

- Caching mechanisms
- Faster scraping techniques
- Model optimization

10 Security Enhancements

- API authentication (JWT)
- Secure data handling

Appendix

Appendix A: Sample API Endpoints

Method	Endpoint	Description
GET	/healthz	Health check
POST	/research	Start research
GET	/research/{id}	Get result
GET	/history	Fetch reports
POST	/search	Semantic search

Appendix B: Sample Docker Commands

```
docker compose up -d
docker compose down
docker image prune -af
```

Appendix C: Project Structure

```
artifacts/
├── python-backend/
├── api-server/
└── energy-researcher/
```

Bibliography

[1] T. Brown et al., “Language Models are Few-Shot Learners,” Advances in Neural Information Processing Systems, 2020.

[2] Docker Inc., “Docker Documentation,” Available: <https://docs.docker.com/>

[3] Amazon Web Services, “Amazon EC2 Documentation,”

Available: <https://docs.aws.amazon.com/ec2/>

[4] DuckDuckGo, “DuckDuckGo Search API,” Available: <https://duckduckgo.com>